Database Setup Guide

IIT Ropar Purchase Indent System (PIS) - PostgreSQL + Prisma, from zero to a running database.

1. What you are setting up

The backend stores everything in PostgreSQL. We never write SQL by hand for tables. We describe tables as "models" in one file, backend/prisma/schema.prisma, and Prisma creates the real tables from it.

- PostgreSQL: the database server (runs on your laptop, port 5432).
- Prisma: the tool that reads schema.prisma and creates tables (migrate), and gives the backend code a typed client (generate).
- backend/.env: holds the connection string so the password never goes into git.

2. Install PostgreSQL (Windows)

1. Download the installer from postgresql.org/download/windows (version 16 recommended).
2. Run it. Keep all components ticked (PostgreSQL Server, pgAdmin 4, Command Line Tools).
3. When asked, set a password for the "postgres" superuser. Write it down: you need it in section 4.
4. Keep the port as 5432. Finish the installer.
5. Check it works: open a new terminal and run:

```
psql --version
```

Note: If "psql" is not recognised, add C:\Program Files\PostgreSQL\16\bin to your PATH (Windows search > "Edit the system environment variables" > Environment Variables > Path > New), then open a new terminal.

Mac: brew install postgresql@16 then brew services start postgresql@16. Linux: sudo apt install postgresql then sudo service postgresql start.

3. Create the database

Open a terminal and connect as the postgres user (it asks for the password from step 2):

```
psql -U postgres
```

You are now at a prompt that looks like postgres=#. Type:

```
CREATE DATABASE pis;
\l
\q
```

- CREATE DATABASE pis; makes an empty database called pis.
- \l lists all databases. You should see pis in the list.
- \q quits.

Prefer clicking? Open pgAdmin 4 > Servers > PostgreSQL 16 > right click Databases > Create > Database > name it pis > Save.

4. Connect the backend to it

1. In the backend folder, copy .env.example to .env (if .env does not exist yet).

2. Edit backend/.env and set DATABASE_URL with your own password:

```
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/pis"
```

The pattern is postgresql://USER:PASSWORD@HOST:PORT/DATABASE_NAME.

Note: If your password has special characters like @ or #, replace them with URL codes (@ = %40, # = %23). Never commit .env to git.

5. Install packages and create the tables

Run these from the backend folder (not the project root):

```
cd pis/backend
npm install
npx prisma validate
npx prisma migrate dev --name init
npx prisma generate
```

Command	What it does
npm install	Installs express, prisma, bcrypt, nodemailer, node-cron and the rest.
npx prisma validate	Checks schema.prisma for mistakes. Fix every error it prints before going on.
npx prisma migrate dev --name init	Creates the tables in the pis database and saves a migration file in prisma/migrations/. Commit that folder.
npx prisma generate	Builds the Prisma Client the backend code imports. migrate dev already runs it, so you only run it again after changing the schema without migrating.

6. Using the SQL files (alternative to Prisma migrate)

You get two files with this guide. Use this route OR section 5, never both on the same database (the second one fails with "relation already exists").

File	What it does
pis_schema.sql	Creates the 2 enums and all 13 tables with foreign keys and indexes. Run this first.
pis_sample_data.sql	Optional. Adds a few departments, budget heads, an admin and one pending user. Run it second.

Option A: psql (terminal)

1. Put both .sql files in one folder, for example C:\pis_sql.
2. Make sure the pis database exists (section 3).
3. Run the files in this order:

```
psql -U postgres -d pis -f C:\pis_sql\pis_schema.sql
psql -U postgres -d pis -f C:\pis_sql\pis_sample_data.sql
```

4. Check the result:

```
psql -U postgres -d pis
\dt
SELECT name, role, status FROM "User";
\q
```

\dt should list 13 tables. No error text and a plain prompt back means success.

Option B: pgAdmin (clicking)

1. Open pgAdmin 4 > Servers > PostgreSQL 16 > Databases > pis.
2. Right click pis > Query Tool.
3. Click the folder icon (Open File), choose pis_schema.sql, then press F5 (Execute).
4. Look for "Query returned successfully" at the bottom. Then repeat with pis_sample_data.sql.
5. Refresh: right click Tables > Refresh. Open pis > Schemas > public > Tables to see all 13.

If something goes wrong

- "type Role already exists" or "relation already exists": the tables are already there. To start clean, drop and recreate the database (this DELETES all data):

```
psql -U postgres -c "DROP DATABASE pis;"
psql -U postgres -c "CREATE DATABASE pis;"
```

- The whole schema file runs as one transaction. If any line fails, nothing is created, so you can fix and re-run safely.
- Change the sample admin password: the file has REPLACE_WITH_BCRYPT_HASH. The backend seed should write a real bcrypt hash instead; do not put a plain password there.

Note: The backend uses Prisma Client. If you build tables with the SQL files, the tables must match schema.prisma exactly, or queries fail. For the real project, prefer section 5 (migrate). Use the SQL files to read the design, to test quickly, or as a backup plan.

7. Look at what was created

Option A: Prisma Studio (easiest, in the browser)

```
npm run db:studio
```

Opens http://localhost:5555 with every table. You can read, add, edit and delete rows. Good for checking data while testing.

Option B: psql

```
psql -U postgres -d pis
\dt
\d "User"
SELECT * FROM "User";
\q
```

- \dt lists tables. \d "User" shows the columns of one table.
- Table names are in quotes because Prisma keeps the capital letters (User, Indent).

Option C: pgAdmin

Servers > PostgreSQL 16 > Databases > pis > Schemas > public > Tables.

8. Load starting data (seed)

An empty database is no use for login. The seed scripts add the first users, lists and form templates:

```
npm run db:seed
npm run db:seed:templates
```

Note: prisma/seed.js and seedTemplates.js are still near-empty starters. Whoever owns them fills them in: seed.js creates the Admin user (password hashed with bcrypt), departments and budget heads; seedTemplates.js loads shared/forms/*.json and shared/workflows/standard.json into FormTemplate and WorkflowTemplate.

9. How to add a table (the daily workflow)

Always the same 4 steps. Example: adding the Attachment table.

1. Open backend/prisma/schema.prisma and add a model:

```
model Attachment {
  id         Int      @id @default(autoincrement())
  indentId   Int
  indent     Indent   @relation(fields: [indentId], references: [id])
  fileName   String
  filePath   String
  uploadedBy Int
  uploadedAt DateTime @default(now())
}
```

2. Add the other side of the relation inside the Indent model (Prisma requires both sides):

```
attachments Attachment[]
```

3. Check it, then migrate with a clear name:

```
npx prisma validate
npx prisma migrate dev --name add_attachment
```

4. Open Prisma Studio and confirm the new table exists. Commit schema.prisma and the new folder in prisma/migrations/ together.

Quick syntax reference

In schema.prisma	Meaning
Int, String, Boolean, DateTime, Json	Column types (Json is useful for form answers).
@id @default(autoincrement())	Primary key that counts up by itself.
@unique	No two rows may share this value (email).
@default(now())	Fills the current time automatically.
String?	The question mark makes the column optional (can be empty).
@relation(fields: [a], references: [id])	Foreign key: column a points to the id of another table.
Attachment[]	The "many" side of a relation. Has no column; Prisma uses it for joins.
@@unique([a, b])	The pair (a, b) must be unique.

10. Tables in this project

Our design has 13 tables. The starter schema.prisma has the first 8; the last 5 still have to be added using the steps in section 9.

Table	Stores	Status
User	Name, email, hashed password, role, department, status	In starter

ListItem	Dropdown values: departments, budget heads, vendors	In starter
FormTemplate	Form JSON per code and version (SPS-101 etc.)	In starter
WorkflowTemplate	Stages and who approves at each stage	In starter
FormLimit	Admin-set limits (amount, quotation count, etc.)	In starter
Indent	One purchase indent: stage, holder, total, answers (Json)	In starter
IndentHistory	Every action taken on an indent (the timeline)	In starter
PurchaseOrder	PO number, vendor, delivery date, status	In starter
ApprovalAuthority	Who can approve up to what amount	To add
Attachment	Uploaded files per indent	To add
PoExtension	Delivery extension requests and approvals	To add
Reminder	Scheduled reminder emails (node-cron reads this)	To add
AuditLog	Admin and security actions	To add

11. Changing or fixing the schema

- Changed a model? Run npx prisma migrate dev --name what_changed. Never edit files inside prisma/migrations/ by hand.
- Pulled a teammate's change? Run npx prisma migrate dev. It applies their new migration files to your database.
- Messed up your local data and want a clean start? This DELETES all data in the pis database:

```
npx prisma migrate reset
```

- Adding a required column to a table that already has rows fails. Make it optional (String?) or give it a @default value.

12. Troubleshooting

Error	Fix
P1001: Can't reach database server	PostgreSQL is not running. Windows: Services > postgresql-x64-16 > Start. Also check the port in DATABASE_URL.
P1000: Authentication failed	Wrong password in DATABASE_URL. Test it with psql -U postgres.
P1003: Database pis does not exist	Run CREATE DATABASE pis; (section 3).
Environment variable not found: DATABASE_URL	.env is missing or in the wrong folder. It must be backend/.env, and you must run commands from backend/.
"psql" is not recognized	Add the PostgreSQL bin folder to PATH (section 2).
npx prisma validate shows errors	Read the line number it prints. The usual cause is a relation missing its other side (section 9, step 2).
EPERM / file locked on generate (Windows)	Stop the running backend (nodemon), then run npx prisma generate again.
Cannot find module @prisma/client	Run npx prisma generate.

13. Checklist

- PostgreSQL installed, psql --version works
- Database pis created
- backend/.env has the right DATABASE_URL
- npm install done in backend/
- prisma validate has no errors
- prisma migrate dev ran and tables show in Prisma Studio
- Seed scripts run